L03. Combinational Logic (2)

Lecture 03

3.1 부울식을 회로로 만들기

> **부울식 ↔ 회로도 직접 대응. 식 하나가 여러 게이트 구성으로 나옴.**

- Z = A·(B·C) = A(BC) = A·B·C 이면 2-입력 AND 2개, 아니면 3-입력 AND 1개.
- IC/VLSI 는 라이브러리 게이트(미리 설계·검증된 셀)로 구현. 식을 쓸 수 있는 게이트에 대응시킴.

Technology mapping = 부울 네트워크를 쓸 수 있는 라이브러리 게이트로 바꿔 표현하는 과정.

라이브러리 게이트	미리 설계된 셀(AND, OR, NAND, XOR …).
왜 중요한가	부울식 하나를 게이트에 대응시키는 방법이 여럿. 최선의 구현 찾기가 설계의 일.

- 설계 = 논리 최소화 + technology mapping.

한 함수의 서로 다른 구현

Z = A′B′C + A′BC + AB′C + ABC′ = Σm(1, 3, 5, 6), 구현 세 가지.

A	B	C	Z
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1

A	B	C	Z
1	1	1	0

식	구현	특징
$Z_1 = ABC' + A'C + B'C$	2-level 구현	단 수가 2
$Z_2 = ABC' + (AB)'C$ ($(AB)'C = A'C + B'C$, De Morgan)	multi-level 구현	게이트 fan-in 이 줄어듦
$Z_3 = (AB) \oplus C$	복합 게이트 XOR 사용	게이트 수가 가장 적음 (개별 부품이라면. VLSI 에서는 아닐 수 있음)

게이트 플레이그라운드: 같은 함수의 서로 다른 식

ABCF00000011010001111000101111011110

게이트: NOT 3, AND 3, OR 1

입력 A=0, B=0, C=0 → F = 0

진리표 기준 최소 SOP: $ABC' + A'C + B'C$

비교 식 G = (AB)^C — 동치: 모든 입력에서 F = G.

논리 블록과 계층

복잡한 함수도 기본 게이트를 모아서 만듦. 계층 설계: half adder → full adder → 2-bit adder.

3.2 2단 논리 (Two-level logic)

- **회로 level**: 입력에서 출력까지 경로상 게이트 수의 최댓값. 입력단 인버터는 안 셈(입력의 true/complement 형이 둘 다 있다고 가정). 경로 level = 경로 위 논리 게이트 수, 회로 level = 경로 level 의 최댓값.

용어

용어	정의	예 / 판별
변수	A (A′), B (B′), C (C′)	Z = AB′C + A′B + A′BC′ + B′C : 변수 3개
literal	식에 나오는 변수 또는 그 보수의 매번의 출현	위 식은 리터럴 10개
product term	변수의 AND	AB′C ◯, (A+B)C ✕
SoP	곱항의 합. AND-OR 2단 회로	AB′ + CD′E ◯, (A+B)C + D′E ✕
sum term	변수의 OR	A+B′+C ◯, AB+C ✕
PoS	합항의 곱. OR-AND 2단 회로	(A+B′)(C+D′+E) ◯, (AB+C)(D′+E) ✕

- SoP ↔ PoS 는 multiplying과 factoring로 변환. (A+B)(C+D) [PoS] ⇔ AC + AD + BC + BD [SoP]. 바꾸면 회로도도 달라짐.
- 2단 논리는 true 입력(A, B, C …)과 complement 입력(A′, B′, C′ …)을 모두 쓰면 어떤 함수든 구현 가능.

3.3 정준형 (Canonical form)

> 정준형 = 함수의 표준 형태, unique algebraic signature. 최소형은 아님.

- 진리표가 부울 함수의 유일한 서명. 두 조합 시스템이 같은 기능인지 확인 가능하지만 행이 너무 많아서 큰 시스템엔 불가(복잡도).
- 일반 부울식은 편리하지만 같은 진리표라도 식(게이트 구현)이 여럿 나옴.
- 정준형은 최소형이 아니어도 유일한 서명. 진리표에서 minterm 전개나 maxterm 전개로 얻음.

Minterm 과 maxterm

minterm	각 변수가 true 또는 complement 형으로 정확히 한 번(둘 다는 안 됨) 나오는 리터럴의 곱. 3변수 예: ABC ◯, A′BC′ ◯, (A+B)C ×, ABA′C ×, AB ×.
maxterm	각 변수가 정확히 한 번 나오는 리터럴의 합. 예: A+B+C ◯, A′+B+C′ ◯, AB+C ×, A+B+A′+C ×, A+B ×.

- n입력이면 minterm 2^n개, maxterm 2^n개. 진리표 행 수와 같음.
- minterm ⊆ 곱항, sum-of-minterms ⊆ SoP. maxterm ⊆ 합항, product-of-maxterms ⊆ PoS.
- minterm = 큐브의 꼭짓점 하나.

> 행 하나당 minterm 하나: 그 행에서만 1. maxterm 은 그 행에서만 0.

A	B	minterm	index	maxterm	index
0	0	A′B′	m_0	A+B	M_0
0	1	A′B	m_1	A+B′	M_1
1	0	AB′	m_2	A′+B	M_2
1	1	AB	m_3	A′+B′	M_3

- A′B′ 는 A = 0, B = 0 일 때만 1, 나머지는 0. A+B 는 A = 0, B = 0 일 때만 0, 나머지는 1.

Minterm 전개

- 출력이 1인 행의 minterm 을 뽑는다.
- 뽑은 minterm 전부 OR. canonical sum(sum-of-minterms, SoP). AND-OR 2단 회로로 구현.

예 · 2변수

A	B	F
0	0	0
0	1	0
1	0	1
1	1	0

F 는 A=1, B=0 일 때만 1. F = AB′

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

F 는 A′B, AB′ 어느 쪽이든 1. F = A′B + AB′

예 · 3변수

A	B	C	F	F′	minterm		maxterm	
0	0	0	0	1	A′B′C′	m_0	A+B+C	M_0
0	0	1	1	0	A′B′C	m_1	A+B+C′	M_1
0	1	0	0	1	A′BC′	m_2	A+B′+C	M_2
0	1	1	1	0	A′BC	m_3	A+B′+C′	M_3
1	0	0	0	1	AB′C′	m_4	A′+B+C	M_4
1	0	1	1	0	AB′C	m_5	A′+B+C′	M_5
1	1	0	1	0	ABC′	m_6	A′+B′+C	M_6
1	1	1	1	0	ABC	m_7	A′+B′+C′	M_7

minterm 은 정확히 한 행만 덮음. maxterm 은 한 행(0)만 빼고 나머지 행(1)을 전부 덮음.

F = A′B′C + A′BC + AB′C + ABC′ + ABC (= AB + C)

F′ = A′B′C′ + A′BC′ + AB′C′

F(A,B,C) = Σm(1,3,5,6,7) = m_1+m_3+m_5+m_6+m_7, F′(A,B,C) = Σm(0,2,4)

minterm 전개가 최소형이라는 보장은 없음.

Maxterm 전개

- 출력이 0인 행의 maxterm 을 뽑는다.
- 뽑은 maxterm 전부 AND. canonical product(product-of-maxterms, PoS). OR-AND 2단 회로로 구현.

예 · XOR (F = A⊕B)

A	B	F	maxterm
0	0	0	A+B
0	1	1	A+B′
1	0	1	A′+B
1	1	0	A′+B′

F 는 A+B = 0 이거나 A′+B′ = 0 이면 0. F = (A+B)(A′+B′)

예 · 3변수

F = (A+B+C)(A+B′+C)(A′+B+C), F(A,B,C) = ΠM(0,2,4) = $M_0 \cdot M_2 \cdot M_4$

F′ = (A+B+C′)(A+B′+C′)(A′+B+C′)(A′+B′+C)(A′+B′+C′), F′ = ΠM(1,3,5,6,7)

정준형 사이의 변환

변환	방법	예
minterm → maxterm	minterm 번호를 쓰지 않은 번호로 바꿔 maxterm 으로 쓴다	F = Σm(1,3,5,6,7) = ΠM(0,2,4)
maxterm → minterm	maxterm 번호를 쓰지 않은 번호로 바꿔 minterm 으로 쓴다	F = ΠM(0,2,4) = Σm(1,3,5,6,7)
F 의 minterm → F′ 의 minterm	F 에서 쓰지 않은 번호를 나열	F = Σm(1,3,5,6,7) → F′ = Σm(0,2,4)
F 의 minterm → F′ 의 maxterm	F 와 같은 번호를 maxterm 으로 쓴다	F = Σm(1,3,5,6,7) → F′ = ΠM(1,3,5,6,7)

대수적으로 전개하기

- 일반 스위칭 식 → minterm(maxterm) 전개: 진리표에서 읽거나, 대수적으로 x + x′ = 1 을 써서 빠진 리터럴을 채움.

---

f(a,b,c,d) = a′(b′ + d) + acd′ 의 minterm 전개

x + x′ = 1 로 빠진 리터럴을 채움(redundant 항 추가).

f = a′b′c′d′ + a′b′c′d + a′b′cd′ + a′b′cd + a′bc′d + a′bcd + abcd′ + ab′cd′

(0000, 0001, 0010, 0011, 0101, 0111, 1110, 1010)

f = Σm(0,1,2,3,5,7,10,14) 이므로 f = ΠM(4,6,8,9,11,12,13,15)

---

maxterm 전개를 직접 구하기

f 를 합의 곱으로 인수분해하고 xx′ = 0 으로 빠진 리터럴을 채움.

f = a′(b′+d) + acd′ = (a′ + cd′)(a + b′ + d) = (a′+c)(a′+d′)(a+b′+d)

= (a′+b+c+d)(a′+b′+c+d)(a′+b+c+d′)(a′+b′+c+d′) · (a′+b+c′+d′)(a′+b′+c′+d′)·(a+b′+c+d)(a+b′+c′+d)

(각 항은 1000, 1100, 1001, 1101, 1011, 1111, 0100, 0110 에 해당)

= ΠM(4,6,8,9,11,12,13,15)

등식이 성립하는지 확인하기

a′c + b′c′ + ab = a′b′ + bc + ac′ 증명: 양변을 minterm 으로 전개해 비교.

좌변 = a′c(b+b′) + b′c′(a+a′) + ab(c+c′) = $m_3 + m_1 + m_4 + m_0 + m_7 + m_6$

우변 = a′b′(c+c′) + bc(a+a′) + ac′(b+b′) = $m_1 + m_0 + m_7 + m_3 + m_6 + m_4$

양쪽 다 Σm(0,1,3,4,6,7). 같음.

두 함수의 곱

> 서로 다른 minterm 의 곱은 0. i ≠ j 이면 $m_i m_j = 0$. n = 3 이면 $m_1 m_3$ = (A′B′C)(A′BC) = 0.

- $f_1 = \Sigma\, a_i m_i$, $f_2 = \Sigma\, b_j m_j$ 이면 $f_1 f_2 = \Sigma\, a_i b_i m_i$ (i ≠ j 이면 $m_i m_j = 0$). 두 전개에 공통인 minterm 만 남음.

- 예: $f_1 = \Sigma m(0,2,3,5,9,11)$, $f_2 = \Sigma m(0,3,9,11,13,14) \rightarrow f_1 f_2 = \Sigma m(0,3,9,11)$.

3.4 불완전하게 명시된 함수 (don’t care)

- 입력 조합이 안 들어오거나 출력이 안 쓰이는 경우가 있음.
- 이때 출력을 X(don’t care)로 표시.
- X 는 0 으로 보든 1 로 보든 상관없음. 구현할 때 식이 간단해지는 값으로 정함.

- 예: BCD(binary-coded decimal) 입력이면 1010_2 ~ 1111_2 는 안 나옴.

예 · F(A,B,C)

A	B	C	F
0	0	0	1
0	0	1	X
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	X
1	1	1	1

X 의 값	식
두 X 모두 0	F = A′B′C′ + A′BC + ABC = A′B′C′ + BC
첫 X = 1, 둘째 X = 0	F = A′B′C′ + A′B′C + A′BC + ABC = A′B′ + BC
두 X 모두 1	F = A′B′C′ + A′B′C + A′BC + ABC′ + ABC = A′B′ + BC + AB